ADVANCED WEB TECHNOLOGIES · PROJECT REVIEW

EcoTrace

Website guide and review FAQ

How every part of the website works, what happens behind each screen, and answers to the questions reviewers commonly ask.

Team member	Registration No.
Mohammed Afaan Sherief	24MID0178
Amit Kumar	24MID0155
Bhumi Kumari	24MID0216

Stack: HTML, CSS, JavaScript, React (Vite), Node.js, Express, MongoDB, Python (FastAPI, scikit-learn, XGBoost, statsmodels, SHAP, PuLP).

Contents

1. EcoTrace in one page
2. Architecture and how a request flows
3. Signing in, roles and security
4. How a shipment's CO_2, cost and transit time are calculated
5. The website, page by page
6. The machine-learning service
7. Alerts, reports, scheduled jobs and live updates
8. Data: database collections and the demo dataset
9. Results against the success criteria
10. Suggested demo flow for the review
11. Frequently asked questions

Appendix A. REST API reference

Appendix B. Running the project

1. EcoTrace in one page

The problem. Small and medium businesses (MSMEs) ship goods every day, mostly by road, but have no affordable tool to measure, compare, forecast or reduce the carbon emissions of those shipments. Large clients reporting under SEBI's BRSR framework now ask their suppliers for this data.

What EcoTrace does. A business logs or imports its shipments. EcoTrace calculates each shipment's CO_2, shows where emissions come from, compares vendor and transport-mode options, forecasts future emissions, predicts which shipments will be late, flags wrongly entered records, and recommends the cheapest shipping plan that stays within a monthly carbon budget. It produces monthly PDF reports and emails budget alerts.

What the website includes

Area	What it gives the user
Public landing page	The full problem statement, solution, modules, ML components, architecture, tech stack, feasibility and team
Login and roles	Company accounts with Owner, Logistics Manager and Admin roles
Emissions dashboard	KPIs, monthly CO_2 by mode, carbon-budget bar, 3-month forecast, top vendors, route map, high-risk shipments
Shipments	Log one shipment with a live CO_2 preview, bulk CSV import, filters, mark delivered, delay-risk badges, anomaly flags
Vendors and locations	Suppliers, factory, warehouses and customers, geocoded and shown on a map
Option comparator	Every vendor + mode option ranked by carbon, cost and time with user-set priorities
Carbon-budget optimizer	Cheapest plan that meets demand, capacity and deadlines within a CO_2 budget (linear programming)
What-if simulator	Move a share of freight between modes and see CO_2, cost and time change instantly
Supply-chain graph	Vendors → factory / warehouse → customers, links sized and coloured by CO_2
ML insights	Forecast with backtest, delay-model metrics, SHAP reasons, anomaly list, vendor segments
Reports and alerts	Monthly sustainability PDF, email delivery, budget alerts, alert history
Settings	Company profile and budget, editable emission factors, team management

> Every module in the proposal, including all Should-have and Could-have items (SHAP, clustering, what-if, supply-chain graph, Socket.io), is implemented and working.

2. Architecture and how a request flows

EcoTrace has three running services and one database:

Part	Technology	Responsibility	Port
Frontend	React 18 + Vite, React Router, Context API, Axios, Recharts, React-Leaflet, React Flow, Socket.io client	All screens; talks only to the Express API	5173
API	Node.js, Express, Mongoose, JWT, bcrypt, Multer, csv-parser, pdfkit, Nodemailer, node-cron, Socket.io	Auth, CRUD, CO_2 calculation, aggregations, CSV import, reports, alerts, jobs, real-time events	5050
ML service	Python, FastAPI, pandas, scikit-learn, XGBoost, statsmodels, SHAP, PuLP, joblib, pymongo	Forecasting, delay risk, explanations, anomalies, clustering, optimization	8000
Database	MongoDB (Atlas, or a local instance started automatically)	All persistent data	27018 local

Request flow, step by step

1. The React page calls the API with Axios, e.g. GET /api/analytics/summary. An interceptor adds the JWT as an Authorization: Bearer header.
2. Express runs the requireAuth middleware: it verifies the token's signature and expiry, loads the user, and attaches req.user and req.companyId. Routes that change data also run requireRole(...).
3. The route queries MongoDB through Mongoose, always filtered by the user's company, so one company can never see another's data.
4. For ML features, Express calls the FastAPI service over internal REST with a shared secret in the x-api-key header. The ML service reads its training data directly from MongoDB with pymongo.
5. The JSON response returns to React, which renders charts, tables and maps. When data changes, the API also emits a Socket.io event so teammates' screens refresh.

> React never talks to MongoDB or the ML service directly. This keeps secrets (database URI, ML key, OpenRouteService key) on the server only.

Graceful fallbacks

- **ML service offline:** the forecast falls back to a seasonal-naive method and delay risk to a rule-based score, so the dashboard still works. Optimizer, anomaly scan and clustering show a clear "ML service is not reachable" message.
- **No OpenRouteService key or API down:** distance = straight-line (haversine) × 1.25 road factor, and cities come from a built-in table of about 45 Indian cities.
- **No SMTP settings:** emails are logged to the server console instead of sent.
- **No MONGO_URI:** the API starts a local MongoDB automatically and keeps data in server/.data.

3. Signing in, roles and security

Registration and login

- **Register** creates three things: the company profile, the owner account, and (if a city is given) a factory location geocoded from that city.
- **Passwords** are hashed with bcrypt (cost factor 10) and never stored or returned in plain text; the password hash is stripped from every JSON response.
- **Login** returns a JWT signed with a server secret, valid for 7 days. The token holds the user id, company id and role.
- **Session handling:** the token is kept in the browser's localStorage. Any 401 response signs the user out and returns them to the login page.

Roles and permissions

Action	Owner	Logistics Manager	Admin
View dashboard, shipments, vendors, analytics, ML, reports	Yes	Yes	Yes
Log / import shipments, mark delivered	Yes	Yes	Yes
Add and edit vendors and locations	Yes	Yes	Yes
Delete shipments, vendors, locations	Yes	No	Yes
Change company profile, carbon budget, alert email	Yes	No	Yes
Edit emission factors	Yes	No	Yes
Add / remove team members	Yes	No	Yes (cannot add or remove an owner)
Retrain ML models	Yes	No	Yes

Permissions are enforced on the server by requireRole middleware; the frontend only hides buttons for convenience.

Other safeguards

- Every query is scoped by company id (multi-tenant isolation).
- A vendor or location with shipments cannot be deleted, so history is never orphaned.
- CSV uploads are limited to 5 MB and 5,000 rows; each row is validated and bad rows are reported, not silently dropped.
- Secrets live in .env files (dotenv) and are excluded from version control.
- The Socket.io connection is authenticated with the same JWT, and each user only joins their own company's room.

4. How a shipment's CO_2, cost and transit time are calculated

This is the core of EcoTrace and runs every time a shipment is previewed, saved or imported (server/src/services/shipmentBuilder.js).

Step 1: Find the two places

The origin is a vendor (inbound) or the company's own factory or warehouse (outbound); the destination is a location. Each has coordinates stored as a GeoJSON point, geocoded from its city when it was created.

Step 2: Distance for each transport mode

Mode	How distance is computed	Available when
Road	OpenRouteService driving-hgv (truck) route; without a key, straight-line × 1.25. Cached in MongoDB.	Always
Rail	Road distance × 1.1	Road distance ≥ 250 km
Sea (coastal)	Road distance × 1.4	Both ends are port cities and road distance ≥ 500 km
Air	Great-circle distance × 1.05	Straight-line distance ≥ 300 km

Step 3: Apply the formulas

```
CO₂ (kg)      = distance (km) × weight (t) × factor (kg CO₂e/t-km)
Freight (₹)   = max(min charge, distance × t × ₹ per t-km)
Transit days = ceil(handling days + distance ÷ km per day)
```

Factors used (editable under Settings → Emission factors)

Mode	kg CO_2e / tonne-km	₹ / tonne-km	Min charge ₹	km / day	Handling days
Road	0.107	3.2	1,500	450	0.5
Rail	0.028	1.6	4,000	350	2
Sea	0.016	1.1	6,000	420	3
Air	1.13	28	3,000	6,000	1

Emission factors are approximated from the UK Government GHG Conversion Factors (freighting goods). Costs, speeds and handling times are indicative Indian freight averages.

Worked example (verified in the running system)

Konkan Steel Supply (Mumbai) → Sahyadri Plant (Chakan), 8,000 kg of steel by road:

```
136.8 km × 8 t × 0.107 = 117.10 kg CO₂e
```

The system returned exactly 117.10 kg. For a longer route the difference between modes is striking: Mumbai → Delhi with 8 t emits about 1,234 kg by road, 355 kg by rail and 10,947 kg by air.

Step 4: Delay risk

If the shipment is still in transit, the API asks the ML service for its probability of arriving late, with the top reasons (Section 6.2). The saved shipment stores distance, emission factor, CO_2, cost, expected days and risk. When it is marked delivered, actual days are recorded and it is flagged late if actual > expected.

5. The website, page by page

5.1 Landing page (/)

What the user sees. A public page presenting the whole project: problem, objectives, workflow, nine modules, ML components, an architecture diagram, tech stack tabs, feasibility analysis, conclusion and team.

How it works.

- Static React page; no login needed. “Try the live demo” goes to the login page, which has one-click demo accounts.

5.2 Emissions dashboard (/app)

What the user sees. Five KPI cards (total CO_2e, shipments, carbon intensity in g per tonne-km, freight cost, on-time rate), a stacked area chart of monthly CO_2 by mode, this month’s budget bar, share by mode, the 3-month forecast, top vendors, an interactive route map, high-risk shipments in transit, and top routes. Filters: period (6/12/24 months, all) and mode.

How it works.

- One MongoDB aggregation with $facet computes totals, monthly series, by-mode, by-vendor and by-route breakdowns in a single query. It returns 2 years (~4,900 shipments) in about 80 ms.
- The route map (React-Leaflet on OpenStreetMap tiles) draws each origin–destination pair as a line whose width and shade grow with total CO_2.
- The forecast comes from /api/ml/forecast, cached for 12 hours in the forecasts collection.
- The page refreshes automatically when a teammate logs a shipment (Socket.io).

5.3 Shipments (/app/shipments)

What the user sees. A searchable, filterable, paginated table (mode, status, inbound/outbound, risk level, flagged only, sort by CO_2 / cost / distance). Each row shows route, mode, material, weight, distance, CO_2, cost, delivery status, delay-risk badge and anomaly flag.

How it works.

- **Log shipment:** choose inbound (from a vendor) or outbound (to a customer), destination, mode, weight, material and date. While typing, POST /api/shipments/preview returns distance, CO_2, cost, transit days and delay risk, and shows the calculation as a formula. Modes not available on that route are disabled.
- **Import CSV:** upload a file (template downloadable). Multer receives it in memory, csv-parser reads it, and each row goes through the same calculation. Unknown vendors or destinations are created from their city column. The result shows imported and skipped rows with the reason for each skip.
- **Mark delivered:** enter actual days; the shipment becomes delivered and late if actual > expected.
- **Risk badge:** click it to see the probability and SHAP reasons (e.g. “Vendor late 8/10 times +21%”).

5.4 Vendors (/app/vendors) and Locations (/app/locations)

What the user sees. Vendors list city, materials, price per kg, monthly capacity, shipments, total CO_2, kg CO_2 per tonne delivered, on-time rate and K-Means segment. Locations list the factory, warehouses and customers, with a network map of all sites.

How it works.

- Adding or editing geocodes the city (built-in table first, then OpenRouteService) and stores a GeoJSON point with a 2dsphere index.
- A geospatial $nearSphere query (/api/locations/:id/nearby-vendors) finds vendors within a radius.
- Per-vendor statistics come from the same aggregation as the dashboard.

5.5 Option comparator (/app/compare)

What the user sees. Pick a destination, material and weight, then set priorities with three sliders (carbon, cost, delivery time) or presets (Greenest, Cheapest, Fastest, Balanced). Every vendor × mode option is ranked with a score from 0 to 100 and a recommendation that states the CO_2 saved versus the most carbon-heavy option.

How it works.

- Sources: for a factory or warehouse, vendors that supply the chosen material; for a customer, the company's own factory and warehouses.
- For every source and every available mode it computes distance, CO_2, freight cost, goods cost (vendor price × weight), total cost and days, plus the vendor's historical on-time rate.
- Scoring: each metric is min–max normalised so the best option scores 1 and the worst 0, then combined as a weighted average using the slider weights. Sliders re-rank instantly in the browser without another server call.

$$\text{score} = 100 \times \Sigma\, w_i \cdot n_i \div \Sigma\, w_i \quad \text{for } i = \text{carbon, cost, time}$$

w_i is the slider weight; n_i is the normalised metric (1 = best option, 0 = worst).

5.6 Carbon-budget optimizer (/app/optimizer)

What the user sees. A demand table pre-filled from last month's real shipments (destination, material, total weight, deadline), a CO_2 budget slider, and a "Find cheapest plan" button. The result shows plan cost and CO_2 compared with the current mix, budget used, the carbon shadow price, the share of tonnage by mode, and the allocation table (which source and mode carries how many tonnes).

How it works.

- The API builds every feasible option per demand row (source × mode whose transit time meets the deadline) with a cost per kg and CO_2 per kg, and sends them to the ML service.
- PuLP (CBC solver) solves a linear program (formulation below).
- If the budget is impossible, it solves a second LP that minimises CO_2 and tells the user the lowest achievable value.
- The shadow price of the budget constraint shows how much extra cost each further kg of CO_2 reduction would need.

$$\begin{aligned}
\text{minimise} \quad & \Sigma\ \text{cost_per_kg}(o) \times x(o) \\
\text{subject to} \quad & \Sigma\ x(o),\ o \in \text{options of demand } d \quad = \text{weight}(d) \\
& \Sigma\ x(o),\ o \in \text{options of vendor } v \quad \le \text{capacity}(v) \\
& \Sigma\ \text{co2_per_kg}(o) \times x(o) \quad \le CO_2 \text{ budget} \\
& x(o) \ge 0
\end{aligned}$$

x(o) is the kg sent with option o. The three constraints meet every demand, respect vendor monthly capacity, and keep total CO_2 within the budget.

Options slower than a row's deadline are excluded before solving. Costs are linear (vendor price + tariff per tonne-km); minimum charges are not modelled.

5.7 What-if simulator (/app/what-if)

What the user sees. Sliders such as "Move road freight to rail", "Move road freight to coastal shipping" and "Move air freight to road", with instant CO_2, cost and transit-time changes and a before/after bar chart by mode.

How it works.

- Uses tonne-km totals per mode for the chosen period (/api/analytics/mode-totals). Moving a share of tonne-km from mode A to mode B recalculates CO_2 and cost with each mode's factor, entirely in the browser.

5.8 Supply-chain graph (/app/supply-chain)

What the user sees. An interactive network: vendors on the left, factory and warehouse in the middle, customers on the right. Thicker, darker links carry more CO_2. Clicking a node highlights its links and lists them with CO_2 and trip counts.

How it works.

- Built with React Flow from the by-route aggregation (/api/analytics/supply-chain). Node type comes from the vendor and location collections.

5.9 ML insights (/app/insights)

What the user sees. ML service status, the forecast with its 80% band and backtest table, the delay model's metrics against the baseline, vendor segments, and the anomaly list with a "Scan all shipments" button and a "Mark OK" action for false alarms.

How it works.

- Owners and admins can retrain models; retraining also re-scores all in-transit shipments. Details of each model are in Section 6.

5.10 Reports and alerts (/app/reports)

What the user sees. Choose a month to view, download or email the sustainability PDF; see this month's budget status and the alert history.

How it works.

- The PDF (pdfkit) contains total CO_2e and change versus the previous month, a budget bar, emissions by mode, top vendors, highest-emission routes and written recommendations.

5.11 Settings (/app/settings)

What the user sees. Three tabs: Company and budget, Emission factors, Team and roles.

How it works.

- Changing the budget resets the month's alert level and re-checks it immediately. Changed factors apply to new calculations; existing shipments keep the factor they were calculated with.

6. The machine-learning service

A FastAPI microservice (ml/app). Every endpoint requires the shared API key. Models are trained per company on that company's shipments read from MongoDB, saved with joblib, and retrained weekly by a cron job (or on demand).

6.1 Emission forecasting

- **Data:** total CO_2 per complete calendar month; shipments flagged as anomalies are excluded first so data-entry errors do not distort the forecast.
- **Model selection:** several SARIMA(X) candidates are compared on a rolling backtest and the one with the lowest error is used. On the demo data the winner is SARIMAX(1,0,0) on log CO_2 with Indian season regressors (festival Sep–Nov, monsoon Jun–Aug, January). Prophet with Indian holidays is used automatically if it is installed.
- **Output:** next 3 months with an 80% confidence band.
- **Evaluation:** the last 3 months are hidden, forecast, and compared with the naive "same as last month" baseline on MAE and MAPE.

6.2 Delay-risk prediction

- **Target:** delivered late (actual days > expected days).
- **Features:** route distance, log of weight, expected transit days, monsoon month, festival month, inbound or outbound, transport mode (one-hot), and the vendor's past late rate.
- **No leakage:** the vendor late rate for each training row only uses that vendor's earlier shipments, smoothed toward the overall rate.
- **Models:** Random Forest (300 trees, balanced class weights) and XGBoost are both trained; the higher ROC-AUC wins. The decision threshold is tuned for F1 on a validation split.
- **Evaluation:** a stratified 80/20 hold-out across all seasons, compared with the rule "road shipment in the monsoon will be late".
- **Risk levels:** green under 25%, amber 25–50%, red 50% and above.

6.3 Explainable risk (SHAP)

For every prediction, SHAP's TreeExplainer splits the predicted probability into each feature's contribution. The three largest (risk-raising first) become readable reasons such as "Vendor late 8/10 times +21%", "Route 1,706 km +11%" or "Festival season +6%".

6.4 Anomaly detection (Isolation Forest)

- Three signals per shipment, each compared with similar shipments using a robust z-score (median and MAD): unusually heavy weight for that mode and material, invoiced cost versus the tariff estimate, and distance versus the usual distance on the same route.
- Deviations within 3 robust standard deviations are treated as normal; Isolation Forest then isolates the genuine outliers.
- Each flag gets a reason, e.g. "Invoiced cost Rs 137,916 is 12.7x the tariff estimate" or "Distance 7,922 km is 8.0x the usual for this route". Users can mark a flag as OK and it is not flagged again.

6.5 Vendor segmentation (K-Means)

Vendors are clustered on carbon intensity (kg CO_2 per tonne delivered), log price per kg and on-time rate, after standardising. Clusters are named from their centre compared with the median, e.g. "Green and Reliable" or "Carbon-heavy and Late-prone". The silhouette score is reported.

6.6 Carbon-budget optimizer (PuLP)

The linear program described in Section 5.6, solved with the CBC solver bundled with PuLP.

7. Alerts, reports, scheduled jobs and live updates

Carbon-budget alerts

- After every shipment is saved or imported, and daily at 09:00, the API totals this month's CO_2.
- The first time it passes **80%** of the budget, a warning alert is created; the first time it passes **100%**, a critical alert. Each level fires once per month.
- An alert is saved, pushed live to everyone in the company (bell icon and toast), and emailed to the alert email or the owners.
- After an anomaly scan finds new suspicious records, an anomaly alert is also raised.

Scheduled jobs (node-cron)

When	Job
Every Sunday 02:00	Retrain the ML models for every company and clear cached forecasts
1st of every month 08:00	Build last month's PDF report, email it, and post a "report ready" alert
Every day 09:00	Re-check every company's carbon budget

Real-time updates (Socket.io)

When a user connects, the server verifies their JWT and puts the socket in a room for their company. Creating, importing, updating or deleting shipments, and new alerts, are broadcast to that room. Open dashboards and lists reload and show a toast such as "Arjun Nair logged a shipment: Mumbai to Chakan (117 kg CO_2e)". This was tested with two users: the owner's screen received the manager's new shipment instantly.

8. Data: database collections and the demo dataset

MongoDB collections (Mongoose schemas)

Collection	Holds
companies	Name, industry, city, monthly CO_2 budget, alert email, last budget alert level
users	Name, email, bcrypt password hash, role, company
locations	Factory, warehouses and customers with a GeoJSON point (2dsphere index) and port flag
vendors	City and point, materials, price per kg, monthly capacity, contact, K-Means segment
shipments	Direction, vendor / origin / destination with snapshots, material, mode, weight, distance, factor, CO_2, cost, ship date, expected and actual days, status, late flag, risk, anomaly flag, source
emissionfactors	Per mode: kg CO_2e per tonne-km, cost per tonne-km, minimum charge, speed, handling days, source
forecasts	Cached forecast with history, next 3 months, model name and metrics
alerts	Budget, anomaly, risk and report alerts with level and read status
distancecaches	Cached road distances, so the routing API is not called twice for the same pair

Shipments are indexed on (company, shipDate) and (company, mode) for fast dashboard queries.

The demo dataset

No real company data was available, so EcoTrace seeds a demo company, **Sahyadri Auto Components Pvt Ltd** (an auto-parts maker in Chakan, Pune), with about 4,900 shipments over 2 years, 11 vendors and 10 locations. Deliberate patterns give the models real signal:

- Monsoon (June–September) raises the chance of delay, especially by road.
- Festival season (September–November) raises volumes by about 35%; volumes also grow about 18% over two years.
- Two chronically late vendors (Ludhiana Fasteners and Guwahati Rubber Traders).
- Long routes are slightly more delay-prone; Sundays are quiet.
- 14 deliberately corrupted records (extra zero in the weight, a 12× invoice, a wrong destination) to test anomaly detection.

A separate Python generator (ml/scripts/generate_synthetic.py, pandas + NumPy) produces an equivalent dataset as a CSV in the import format, to demonstrate CSV import.

Demo account	Email	Password
Owner	owner@ecotrace.demo	demo1234
Logistics Manager	manager@ecotrace.demo	demo1234
Admin	admin@ecotrace.demo	demo1234

9. Results against the success criteria

Measured on the demo data with ml/scripts/evaluate.py and the running system.

Success criterion (from the proposal)	Result	Met
CO_2 matches manual calculation	136.8 km × 8 t × 0.107 = 117.10 kg; system returned 117.10 kg	Yes
Dashboard loads 2 years (~5,000 shipments) in under 2 s	Summary aggregation over 2 years: about 80 ms	Yes
Forecast beats naive “same as last month” on MAE / MAPE	MAE 11.8 t vs 18.6 t; MAPE 8.0% vs 12.2% (Jul–Sep 2026 backtest)	Yes
Delay model beats a simple baseline on F1 and ROC-AUC (held-out data)	Random Forest F1 0.564, ROC-AUC 0.761 vs baseline 0.535 / 0.680 (965 held-out shipments). XGBoost: 0.548 / 0.740	Yes
Optimizer meets demand, stays in budget, costs less than current mix	Sept 2026: 17.8% cheaper and 60.8% less CO_2 than the actual mix, all demand met	Yes
Anomaly detection (additional)	All 14 corrupted records found, no false alarms	Yes
Deployed at a public URL	Runs locally; deployment steps for Vercel / Netlify and Render / Railway are documented	Pending

10. Suggested demo flow for the review

About 8–10 minutes. Start all three services first (Appendix B) and open http://localhost:5173.

1. **Landing page:** scroll briefly through problem, modules and architecture.
2. **Sign in as Owner** with the one-click demo button.
3. **Dashboard:** KPIs, CO_2 by mode, budget bar, forecast, route map (hover a line), high-risk shipments.
4. **Shipments → Log shipment:** pick Konkan Steel Supply → Sahyadri Plant, road, 8,000 kg. Show the live CO_2 formula and risk badge. Switch the mode to compare. Save it.
5. **Live update (optional):** have a teammate signed in as the manager on another browser; their screen shows the toast.
6. **Click a red risk badge** to show SHAP reasons.
7. **Compare options:** move the sliders and press Greenest, then Cheapest; the ranking changes.
8. **Optimizer:** lower the budget and press “Find cheapest plan”. Point out savings versus the current mix and the shadow price; try an impossible budget to show the minimum-CO_2 message.
9. **What-if:** move 30% of road freight to rail.
10. **Supply chain:** click the factory node.
11. **ML insights:** backtest table, model comparison, anomaly list with reasons, vendor segments.
12. **Reports:** view the monthly PDF.
13. **Settings → Team:** show the three roles; optionally sign in as the manager to show restricted actions.

11. Frequently asked questions

General and design

Q. What problem does EcoTrace solve, in one sentence?

It lets small businesses measure, compare, forecast and reduce the CO_2 of their shipments, so they can make greener decisions and give clients the supplier emission data they increasingly ask for.

Q. Who are the users?

MSME owners, logistics managers and admins. The outputs (PDF reports) are also meant for the business's large clients who need value-chain (Scope 3) emission data for ESG/BRSR reporting.

Q. How is this different from free online carbon calculators?

Calculators handle one shipment at a time. EcoTrace keeps history, analyses it by mode, vendor and route, compares alternatives, forecasts, predicts delays, detects bad data and optimizes a whole month's plan within a budget.

Q. Which emissions does it cover?

Transport emissions only (upstream and downstream freight). Manufacturing and packaging emissions are out of scope.

Data and calculation

Q. Where do the emission factors come from? Are they accurate?

They approximate the UK Government GHG Conversion Factors for freighting goods (kg CO_2e per tonne-km). They are published averages, so results are estimates, not certified carbon accounting. They are stored in our database and editable, so India-specific factors can replace them.

Q. How is distance calculated?

With an OpenRouteService key, the actual truck route (driving-hgv profile). Without a key, straight-line distance × 1.25. Rail, sea and air derive from these (Section 4). All distances are cached in MongoDB.

Q. Is the data real?

The demo uses synthetic data with realistic built-in patterns, because no real company data was available. Real businesses can import their own records through CSV import, and the models retrain on them.

Q. Why not use a paid carbon API such as Climatiq?

Factors are public and the formula is simple, so storing them locally keeps the system free and working offline. Climatiq was listed as optional.

Q. What happens when a CSV has errors?

Each row is validated independently. Good rows import; bad rows are skipped and listed with the row number and reason (e.g. "rail is not available on this route").

Machine learning

Q. Why did you choose Random Forest over XGBoost?

Both are trained every time and the one with the higher ROC-AUC on held-out data is selected automatically. On our data Random Forest scored 0.761 against XGBoost's 0.740.

Q. How do you know the models are any good?

Each is compared with a simple baseline on data it never saw: the forecast with “same as last month” (MAE 11.8 t vs 18.6 t), the delay model with the rule “road in monsoon = late” (F1 0.564 vs 0.535, ROC-AUC 0.761 vs 0.680).

Q. Why is ROC-AUC only 0.76?

Delays are partly random by nature (and our synthetic data deliberately includes noise), so no model can predict them perfectly. The point is that it clearly beats the baseline and ranks risky shipments well enough to prioritise attention.

Q. Why a random split instead of a time-based split for the delay model?

The most recent 20% of shipments fell almost entirely in the monsoon, which made a time-based test unrepresentative. We use a stratified split across all seasons, and the vendor-history feature only uses earlier shipments, so there is no leakage of future information.

Q. What is SHAP and why use it?

SHAP splits a model’s prediction into each feature’s contribution. It turns a bare probability into reasons a manager can act on, such as “Vendor late 8/10 times +21%”.

Q. Why SARIMA and not Prophet?

Prophet often fails to install (especially on Windows), which was a risk in our proposal. The code uses Prophet automatically if installed; otherwise SARIMA, chosen by backtest from several candidates, with Indian season effects as regressors.

Q. How does anomaly detection decide something is wrong?

It compares each shipment’s weight, invoiced cost and distance with similar shipments using robust z-scores, then Isolation Forest isolates the extreme ones. On the demo data it found all 14 planted errors with no false alarms.

Q. How are models kept up to date?

A weekly cron job retrains them on all shipments, and owners or admins can retrain on demand from the ML insights page.

Optimizer and comparator

Q. Why linear programming rather than just picking the cheapest option?

The cheapest option per row can break vendor capacity or the total CO_2 budget. An LP finds the cheapest combination across all demands that satisfies every constraint at once, and proves it is optimal.

Q. What is the carbon shadow price?

The extra cost of cutting one more kg of CO_2 from the budget. Zero means the budget is not limiting; a positive value shows the cost of tighter targets.

Q. What if the budget is impossible?

The optimizer solves a second problem that minimises CO_2 and reports the lowest achievable value, so the user knows what budget is realistic.

Q. Why ignore minimum charges in the optimizer?

Minimum charges make the problem non-linear (mixed-integer). Keeping it linear keeps it fast and exactly solvable; the current mix is costed on the same basis, so the comparison is fair.

Q. How does the comparator score options?

Each metric (CO_2, total cost, days) is normalised between the best and worst option, then combined with the slider weights into a score from 0 to 100.

Web technology and security

Q. Why MongoDB instead of SQL?

Shipments carry nested snapshots (origin, destination, risk, anomaly) and evolving fields, which fit documents naturally. MongoDB also gives aggregation pipelines for the dashboard and geospatial indexes for locations, both used here.

Q. Why a separate Python service?

The ML libraries (scikit-learn, XGBoost, statsmodels, SHAP, PuLP) are Python. A FastAPI microservice keeps them isolated, independently deployable and reachable only from the API through a shared key.

Q. How is authentication done?

Passwords are hashed with bcrypt. Login returns a signed JWT (7-day expiry) sent with every request; Express middleware verifies it, and role middleware protects sensitive actions.

Q. Can one company see another's data?

No. Every query filters by the company id taken from the verified token, never from the request body.

Q. How do real-time updates work?

Socket.io with JWT-authenticated connections; each user joins their company's room, and the server broadcasts shipment and alert events to that room.

Q. How fast is it and will it scale?

Two years of data (~4,900 shipments) aggregates in about 80 ms thanks to indexes and a single $facet pipeline. Pagination, distance caching, a 12-hour forecast cache and stateless JWT auth help it scale; MongoDB Atlas can scale the database.

Q. What does it cost to run?

₹0: MongoDB Atlas M0, the OpenRouteService free key, Vercel/Netlify and Render/Railway free tiers, and open-source libraries.

Q. Is it deployed?

It currently runs locally. The build is ready for Vercel/Netlify (frontend) and Render/Railway (API and ML), with steps in the README; the API can also serve the built frontend itself.

Limitations and future work

Q. What are the main limitations?

Model accuracy reflects synthetic data; emission factors are averages, so results are estimates; and only transport emissions are covered.

Q. What would you add next?

A pilot with a real MSME's data, weather data (OpenWeather) as a delay feature, India-specific emission factors, live factors from Climatiq, Excel (.xlsx) import, and public deployment.

Appendix A. REST API reference

All endpoints except register and login need a JWT. Base URL: http://localhost:5050/api

Method and path	Purpose
POST /auth/register, POST /auth/login, GET /auth/me	Create company and owner, sign in, current user
GET / POST / DELETE /auth/users	Team management (owner/admin)
GET / POST / PUT / DELETE /vendors, /locations	Vendors and locations CRUD
GET /locations/:id/nearby-vendors	Geospatial search for vendors near a location
GET /shipments	List with filters, search, sort and pagination
POST /shipments/preview	Calculate distance, CO_2, cost, days and risk without saving
POST /shipments, PATCH /shipments/:id, DELETE /shipments/:id	Create, mark delivered or dismiss anomaly, delete
POST /shipments/import, GET /shipments/import/template	CSV import and template
POST /shipments/rescore	Re-score delay risk of all in-transit shipments
GET /analytics/summary	Dashboard totals, monthly, by mode, vendor, route, budget
GET /analytics/supply-chain, GET /analytics/mode-totals	Graph data; what-if inputs
POST /compare	Ranked vendor × mode options
GET /optimize/baseline, POST /optimize	Last month's demand and current mix; run the LP
GET /ml/status, /ml/forecast, /ml/metrics, /ml/anomalies	ML status, forecast, model metrics, flagged shipments
POST /ml/retrain, /ml/anomalies/run, /ml/vendor-clusters, /ml/risk/:id	Retrain, scan, segment, explain one shipment
GET /reports/monthly?month=YYYY-MM, POST /reports/email	PDF report; email it
GET /alerts, POST /alerts/read-all	Alert history
GET / PUT /settings/company, GET / PUT /settings/factors/:mode	Company, budget and emission factors

Internal ML endpoints (FastAPI, port 8000, x-api-key header): /health, /train, /metrics/{company}, /forecast, /delay-risk, /anomalies, /vendor-clusters, /optimize.

Appendix B. Running the project

Prerequisites: Node.js 18+ and Python 3.9+. MongoDB is optional (started automatically if MONGO_URI is empty). Run each service in its own terminal from the project folder:

1. API (port 5050)

```
cd server && cp .env.example .env && npm install && npm start
```

2. ML service (port 8000)

```
cd ml && python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
cp .env.example .env && .venv/bin/uvicorn app.main:app --port 8000
```

3. Frontend (port 5173)

```
cd client && npm install && npm run dev
```

Open http://localhost:5173 and use “Try the live demo”. On the first run the API seeds the demo company automatically.

Useful commands

Reset the demo company:

```
cd server && npm run seed
```

Print the success-criteria metrics:

```
cd ml && .venv/bin/python -m scripts.evaluate
```

Generate a synthetic CSV for the import demo:

```
cd ml && .venv/bin/python scripts/generate_synthetic.py
```

Before the review

- Start all three services 5 minutes early and open the dashboard once, so the forecast is cached.
- Keep this guide and the presentation PDF open in a second window.
- If the ML service fails to start, the dashboard still works with fallbacks; the insights page shows the start command.